



testing_linter

Johan Alexander Peña Ortiz

Cristian Andrés Díaz Ortega

Juan Carlos Manrique Benavides

Martin José Botina Gonzalez

Oscar Leonardo Riveros Perez

Grupo GPT

Universidad Nacional de Colombia

Ambiente y Herramientas

Para la realización del análisis estático del código fuente se utilizaron las herramientas **Pylint** y **Pylint-Django**. La integración de **Pylint-Django** permitió el reconocimiento adecuado de los componentes propios del framework Django, reduciendo la aparición de advertencias incorrectas durante el proceso de análisis.

Con el fin de garantizar la compatibilidad con la estructura del proyecto y adaptar las validaciones a los criterios definidos para el desarrollo, se configuraron las directivas descritas en la **Tabla 1**.

Directiva	Valor configurado	Propósito
init-hook	<code>import os, sys sys.path.insert(0, os.getcwd())</code>	Agrega el directorio raíz del proyecto al path de Python.
load-plugins	<code>pylint_django</code>	Soporte para proyectos desarrollados con Django.
django-settings-module	<code>config.settings</code>	Especifica el módulo de configuración de Django.
missing-module-docstring	Deshabilitada	Evita advertencias por ausencia de documentación a nivel de módulo.
missing-class-docstring	Deshabilitada	Evita advertencias por ausencia de documentación en clases.
missing-function-docstring	Deshabilitada	Evita advertencias por ausencia de documentación en funciones y métodos.

Tabla 1: Reglas aplicadas al linter

Al ejecutar el comando `pylint .\information_app\` se obtiene el resultado que se observa en la **Figura 1**:

[illegible]

Alta	Código duplicado	R0801	Incrementa la deuda técnica y el esfuerzo de mantenimiento.	Extraer la lógica repetida a funciones utilitarias compartidas.
Alta	Imports no utilizados	W0611 / W0614	Añaden ruido al código y dificultan su comprensión.	Eliminar importaciones que no se utilizan.
Media	Función con demasiadas ramas	R0912	Reduce la legibilidad y dificulta futuras modificaciones.	Refactorizar <code>update_equipment()</code> utilizando estructuras de mapeo o funciones auxiliares.
Media	Demasiados argumentos en una función	R0913 / R0917	Hace más difícil entender y mantener la interfaz del método.	Utilizar DTOs, <code>dataclass</code> o diccionarios para agrupar parámetros relacionados.
Media	Demasiados retornos	R0911	Incrementa la complejidad de lectura y mantenimiento.	Dividir la lógica en métodos más pequeños y reutilizables.
Baja	Líneas demasiado largas	C0301	Afecta únicamente la legibilidad.	Dividir expresiones largas en múltiples líneas.
Baja	Espacios en blanco al final de línea	C0303	Problema únicamente de estilo.	Configurar el editor para eliminarlos automáticamente.
Baja	Falta salto de línea al final del archivo	C0304	Problema menor de formato.	Agregar una línea vacía al final del archivo.
Ignorable	Advertencias en migraciones de Django	C0103, C0301, etc.	Son archivos autogenerados y normalmente no se modifican manualmente.	Excluir la carpeta <code>migrations</code> del análisis de pylint.

Tabla 2: Recomendaciones del Linter

Una vez se corrigieron la mayoría de las recomendaciones dadas por el Linter se obtuvo el resultado que se observa en la **Figura 2**.

```
***** Module information_app.repositories.equipment_repository
information_app\repositories\equipment_repository.py:26:4: R0913: Too many arguments (9/5) (too-many-arguments)
information_app\repositories\equipment_repository.py:26:4: R0917: Too many positional arguments (9/5) (too-many-positional-arguments)
***** Module information_app.services.equipment_services
information_app\services\equipment_services.py:99:0: C0301: Line too long (103/100) (line-too-long)
information_app\services\equipment_services.py:163:0: C0301: Line too long (102/100) (line-too-long)
information_app\services\equipment_services.py:272:0: C0301: Line too long (110/100) (line-too-long)
information_app\services\equipment_services.py:73:4: R0912: Too many branches (19/12) (too-many-branches)
***** Module information_app.services.user_services
information_app\services\user_services.py:38:0: R0904: Too many public methods (22/20) (too-many-public-methods)

-----
Your code has been rated at 9.92/10 (previous run: 9.91/10, +0.01)
```

Figura 2: Recomendaciones del linter aplicadas